Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

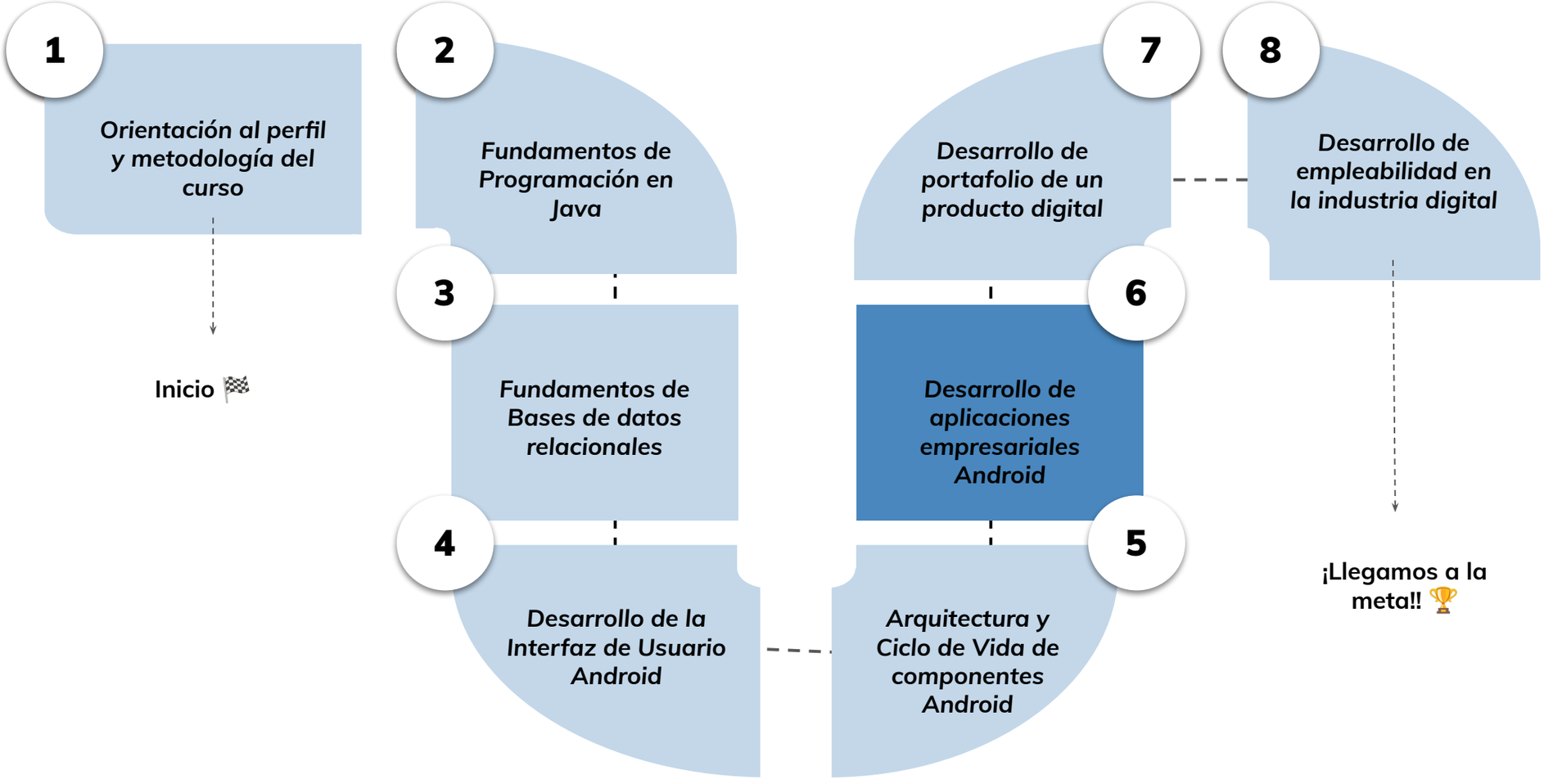
Distribución del aplicativo android -Parte I

AE: Construir un release de un aplicativo android utilizando procedimiento de empaquetamiento para ser distribuido en entornos productivos.

alkemy

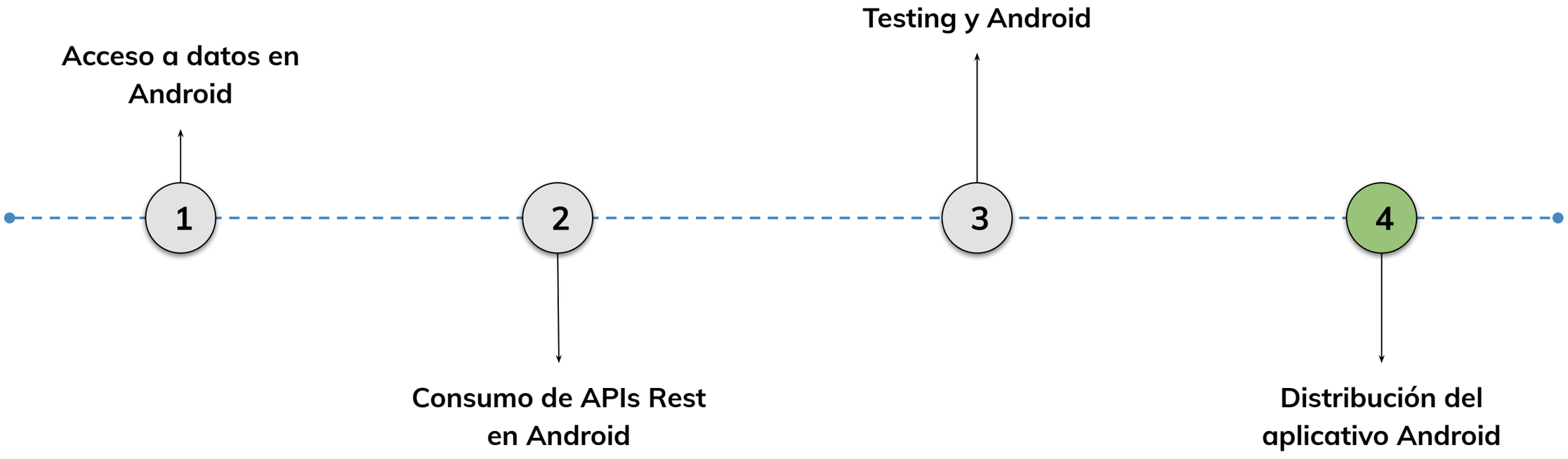
Hoja de ruta

*¿Cuáles **módulos** conforman el programa?*



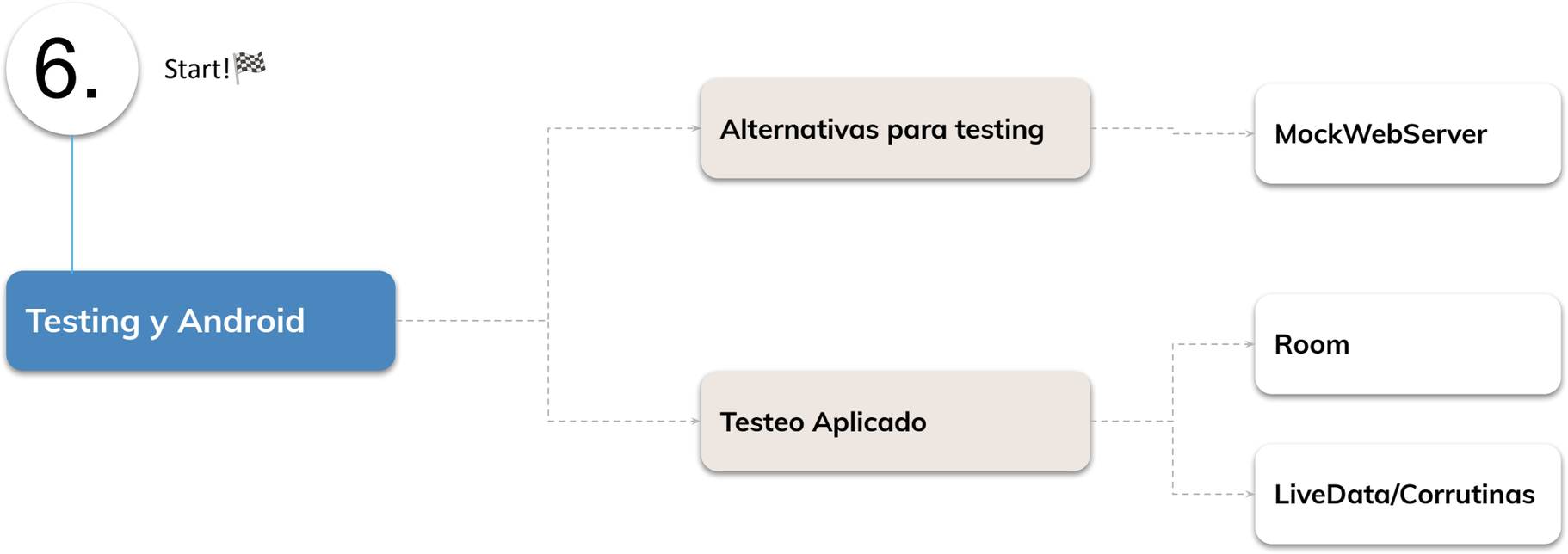
Roadmap de la lección

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



Learning Path

¿Cuáles temas trabajaremos hoy?



alkemy

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

- ✓ *Pasos para agregar tests*
- ✓ *Creación de pruebas*
- ✓ *Implementación de validación de clases*

Learning Path

¿Cuáles temas trabajaremos hoy?

6. Start! 🏁

Distribución del aplicativo android

- **Firma Digital**
  - **Claves**
  - **Certificados**
- **Diferencias**
  - **Apk**
  - **Bundle**

Objetivos de aprendizaje

¿Qué aprenderás?

- *Conocer los pasos para la distribución de aplicación*
- *Saber más de firma digital y su rol en Android*
- *Saber la diferencia entre APK y Bundle*

Rompehielo🥶

Contexto: 🙌

Cuando queramos distribuir al público o a compañeros nuestra aplicación terminada tenemos que crear contenedores que puedan ser enviados e instalado en dispositivos por nuestros compañeros

Consigna: ✍️

¿Cómo creen que podemos probar la identidad de un desarrollador o una compañía al observar estos contenedores?

› Distribución de la aplicación

Distribución de la aplicación

¿Qué es?:
La distribución de una aplicación es el conjunto de acciones y estrategias utilizadas para poner a disposición del público una aplicación móvil. El proceso de distribución implica asegurarse de que la aplicación esté disponible para su descarga e instalación en los dispositivos de los usuarios

Firma Digital

Que es?

es un mecanismo de seguridad que garantiza la autenticidad e integridad del archivo APK (Android Package) de la aplicación.

La firma se crea utilizando una clave privada asociada con un certificado, y el proceso de firma produce una huella digital única que se adjunta al archivo APK

Almacenes de claves, claves y certificados

Los almacenes de claves, claves y certificados son conceptos fundamentales en el ámbito de la seguridad. Un almacén de claves es un archivo seguro utilizado para almacenar y gestionar claves criptográficas, como claves privadas y certificados.

El formato de almacén de claves más común es el JKS (Java KeyStore), que se utiliza para almacenar pares de claves públicas y privadas.

Hay dos tipos principales de claves:

- Privadas
- Publicas

Consideraciones para la firma digital

Es una práctica crucial para mantener la integridad y la confianza de la aplicación. Esto se debe a:

- Actualizaciones automáticas: Cuando una aplicación se firma, en tiendas de aplicaciones, asocia esa firma con la aplicación. Siempre que se publique una actualización hay que asegurarse de que esté firmada con el mismo certificado que la versión anterior. De lo contrario se considera una aplicación nueva y los usuarios no podrán recibir actualizaciones automáticas.
- Compatibilidad con otras aplicaciones y bibliotecas: Firmar la aplicación con el mismo certificado facilita la integración y la interacción con otras aplicaciones y bibliotecas que ya estén utilizando ese certificado como parte de su mecanismo de seguridad o para compartir recursos de manera segura.

LIVE CODING

Ejemplo en vivo

Crear Firmas digitales:

*Vamos a ir al menu Build de Android Studio y despues ir a la opción "**Generate Signed Bundle or APK**". En la ventana que se abre seleccionar APK e ir a "**NEXT**". En la nueva ventana vamos a ver la opción de "**Key store path**" donde el botón "**CREATE NEW**" se encuentra. Clickearla e ingresar datos para crear el certificado*

Tiempo: 15 min

› Diferencias entre APK y Bundle

Diferencias entre APK y Bundle

Un APK (Android Package) es un archivo único que contiene todos los recursos y códigos necesarios para instalar una aplicación en un dispositivo Android.

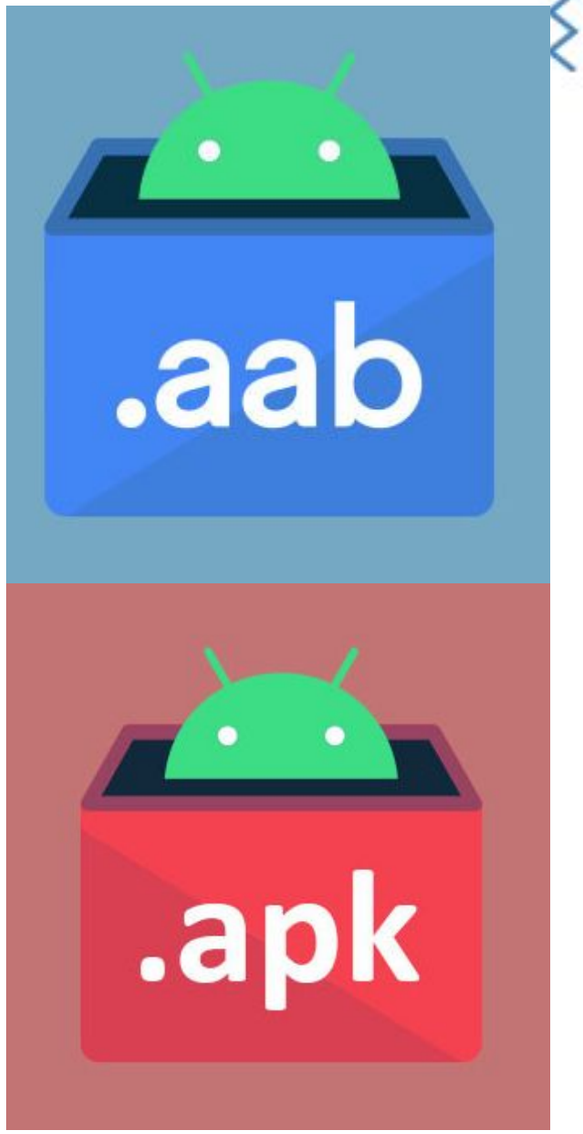
El Android App Bundle es un nuevo formato de publicación que Google Play acepta desde Android 9. Incluye todas las características, recursos y códigos necesarios, pero de forma modularizada.

Creación de APK

Los archivos APK de depuración se generan con el propósito de probar la aplicación durante el desarrollo. Estos archivos se utilizan para realizar pruebas en dispositivos Android y verificar que la aplicación funcione correctamente antes de lanzarla al público.
Los APK de depuración se ejecutarán en tu dispositivo Android y se pueden instalar y probar fácilmente durante el proceso de desarrollo.

Creación de Bundle

- En Android Studio, ve al menú Build y selecciona Generate Signed Bundle / APK.
- En la ventana emergente, elige Android App Bundle y haz clic en Next.
- En la nueva ventana, especifica la ubicación donde deseas que se guarde el Android App Bundle firmado. Puedes utilizar una ubicación predeterminada o seleccionar una personalizada.
- Selecciona Choose existing junto a Key store path y navega para buscar el archivo del almacén de claves de la aplicación. Ingresa la clave de firma y las contraseñas y haz clic en Next.
- En la última pantalla, debes seleccionar el tipo de compilación que deseas utilizar para generar el Android App Bundle firmado. Generalmente, selecciona release para crear un bundle de producción firmado.



LIVE CODING

Ejemplo en vivo

Creamos Bundles y APK firmados:

Una vez que tenemos claves digitales podemos usar el mismo menú que seleccionamos anteriormente y proceder a crear un APK y Bundles firmados. Luego ubicarlos dentro de los archivos del sistema y validar su contenido.

Tiempo: 35 min

¿Alguna consulta?

alkemy

Momento:

Time-out!

⏰5 -10 min.

alkemy

LIVE CODING

Ejemplo en vivo

A continuación veremos los pasos previos para nuestro release

Tiempo: 30 min

BUIL.GRADLE (MODULE:APP)

applicationId

Permite identificar de manera exclusiva la app en el dispositivo y en la Play Store. **NO debe cambiarse una vez publicada la app**.

```
apply plugin:
'com.android.application'
android {
  compileSdkVersion
  29
  buildToolsVersion
  defaultConfig {
    applicationId "com.eme.android.java"
    minSdkVersion 24
    targetSdkVersion 29
    versionCode 1
    versionName "1.0"
  }

  buildTypes
    {  release
    {
      minifyEnabled false
      proguardFiles
getDefaultProguardFile('proguard-android-optimize.txt'),
'proguard-rules.pro'
    }
  }
dependencies
}
}
```

BUIL.GRADLE (MODULE:APP)

versionCode

Valor entero positivo que se usa como número de **versión interna** y se usa para determinar si una versión es más reciente que la otra y **no se muestra a los usuarios**.

```
apply plugin:
'com.android.application'
android {
  compileSdkVersion
  29
  defaultConfig {
    applicationId
    "com.eme.android.java"
    minSdkVersion 24
    targetSdkVersion 29
    versionCode 1
    versionName "1.0"
  }

  buildTypes
    { release
    {
      minifyEnabled false
      proguardFiles
getDefaultProguardFile('proguard-android-optimize.txt'),
'proguard-rules.pro'
dependencies
    }
{ }
  ---
}
```

BUIL.GRADLE (MODULE:APP)

versionName

String que se usa como el número de versión que se muestra a los usuarios (como string sin procesar o como referencia a un recurso de string)

Generalmente se usa versionamiento semántico y su propósito es mostrarse a los usuarios

```
apply plugin:
'com.android.application'
android {
  compileSdkVersion
  29
  buildToolsVersion
  defaultConfig {
    applicationId
    "com.eme.android.java"
    minSdkVersion 24
    targetSdkVersion 29
    versionCode 1
    versionName "1.0"
  }

  buildTypes
    { release
    {
      minifyEnabled false
      proguardFiles
getDefaultProguardFile('proguard-android-optimize.txt'),
'proguard-rules.pro'
    }
  }
}

dependencies {
}
```

NIVEL DE API

compileSdkVersion

minSdkVersion

targetSdkVersion

```
apply plugin:
'com.android.application'
android {
  compileSdkVersion
  29
  buildToolsVersion
  defaultConfig {
    applicationId
    "com.eme.android.java"
    minSdkVersion 24
    targetSdkVersion 29
    versionCode 1
    versionName "1.0"
  }

  buildTypes
    { release
    {
      minifyEnabled false
      proguardFiles
getDefaultProguardFile('proguard-android-optimize.txt'),
'proguard-rules.pro'
    }
  }
dependencies
{ }
---
}
```

TIPOS DE CONSTRUCCIÓN

release

debug

- **applicationIdSuffix**: agregar un segmento al ID de aplicación predeterminado
- **debuggable**: Determina si es posible o no depurar la aplicación. Por defecto es *false*

staging

- **initWith**: Permite copiar configuraciones desde otro tipo de construcción para configurar las opciones que hay que cambiar
- **manifestPlaceholders**: Variable de compilación en el manifiesto

developer.android.com/build-variants

```
android {
  ...

  buildTypes {
    release {
      ...
    }

    debug {
      applicationIdSuffix ".debug"
      debuggable true
    }

    staging {
      initWith debug
      manifestPlaceholders = [hostName:"internal.example.com"]
      applicationIdSuffix ".debugStaging"
    }
  }
}
```

REDUCIR, OFUSCAR Y OPTIMIZAR

Archivos de configuración R8

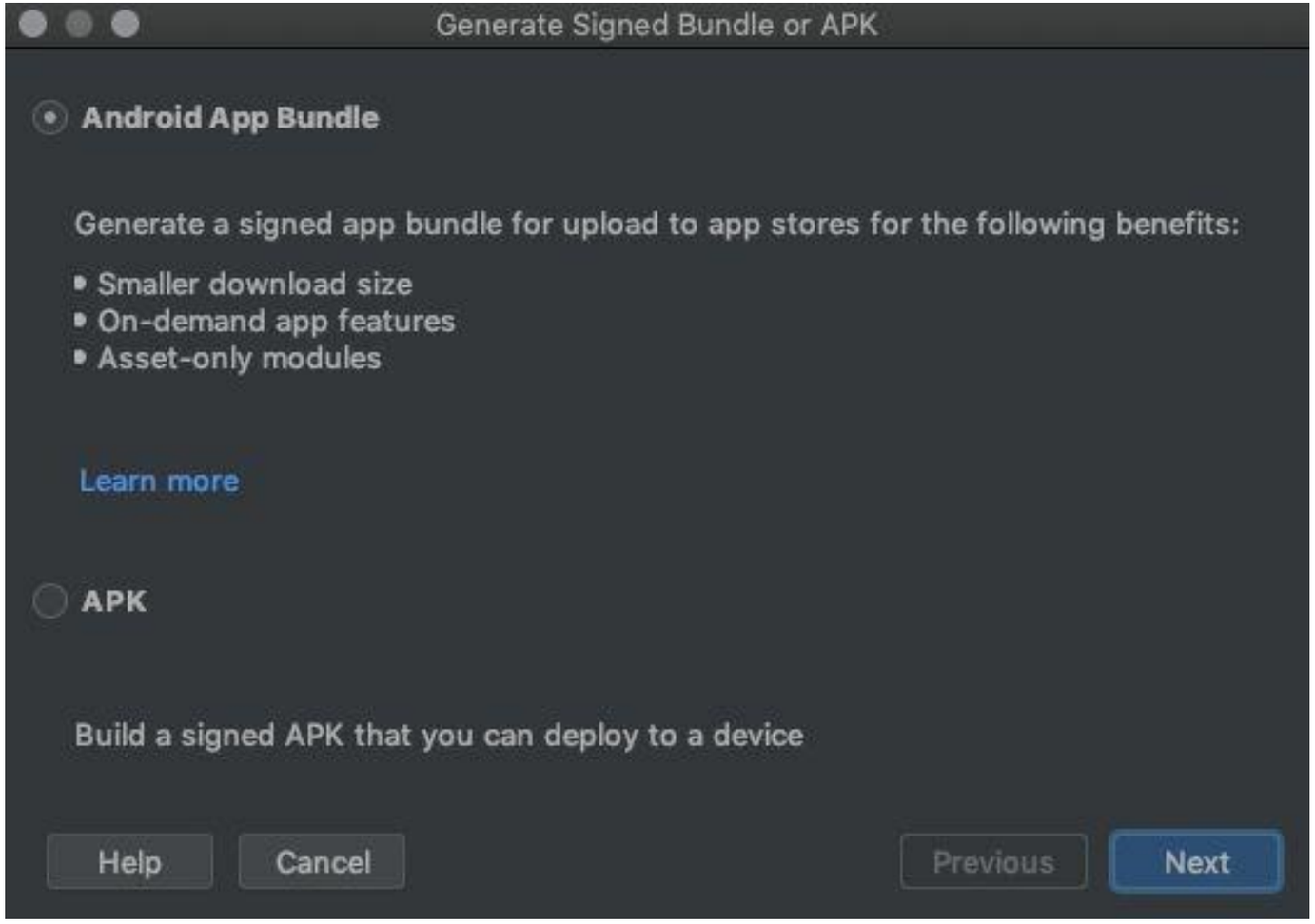
developer.android.com/build-variants

```
apply plugin:
'com.android.application'
android {

  buildTypes
    {  release
    {
        // Permite la reducción de código, la ofuscación y la
        optimización,
        // solo para el tipo de compilación de
        // Habilita minifyEnabled true
        // release de código realizada por Android Gradle
        Plugin  shrinkResources true

        // Incluye los archivos con reglas por defecto de ProGuard que
        vienen
        // con Android Gradle Plugin.
        // Para profundizar, el link a archivos de configuración R8
        proguardFiles
        getDefaultProguardFile('proguard-android-optimize.txt'),
                                        'proguard-rules.pro'
    }

}
```

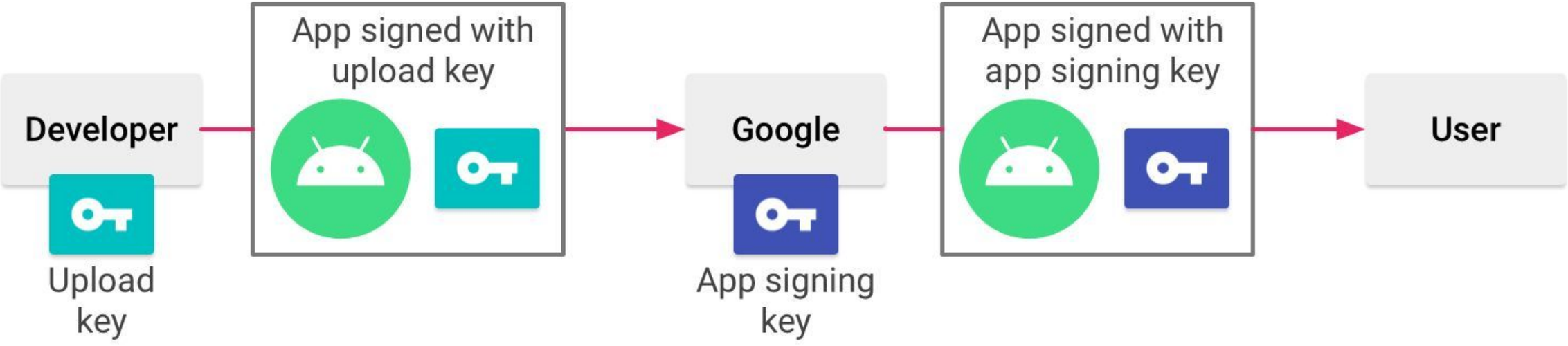
FORMATO DE INSTALADOR

aab (Android App Bundle)

APK (Android Package Kit)



FIRMA DE LA APP



Clave de subida

Clave de firma

developer.android.com/app-signing

FIRMADO DE LA APP

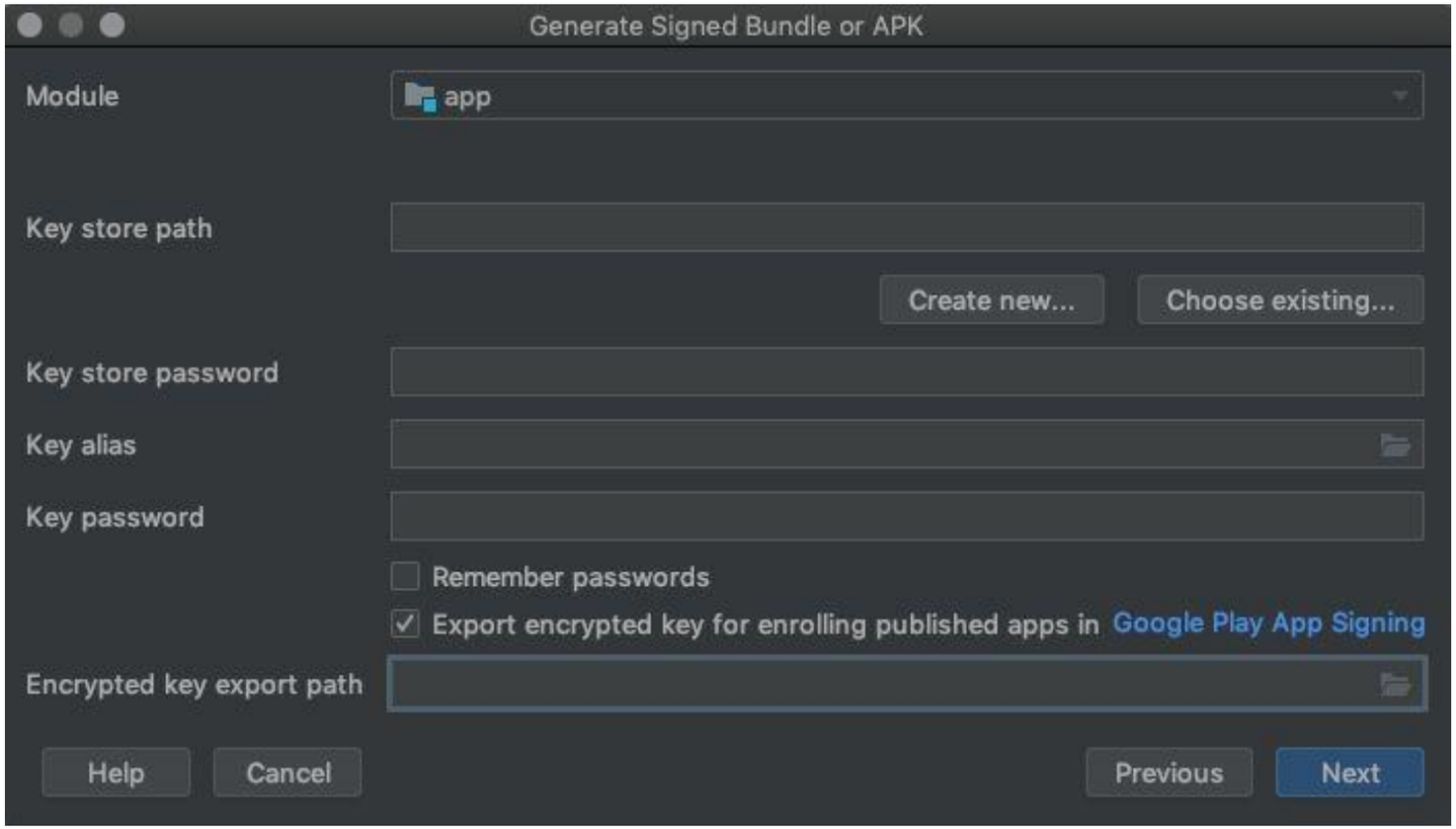
Build → Generate Signed Bundle / APK

Key store

- path: Ubicación del almacén de llaves
- password: crea y confirma una contraseña segura

Key

- alias: Nombre de identificación
- password: crea y confirma contraseña. Debe ser distinta a la del almacén de llaves



TAREAS PREVIAS AL LANZAMIENTO

- Clave criptográfica
- Íconos
- Licencia

→ **Configurar release app**

- Nombre de paquete
- Desactivar depuración
- Limpiar proyecto
- Actualizar manifiesto
- Asignar rutas a servidores
- Instalar un Crashlytics

→ **Compilar release app**

- Compilar con build type de release

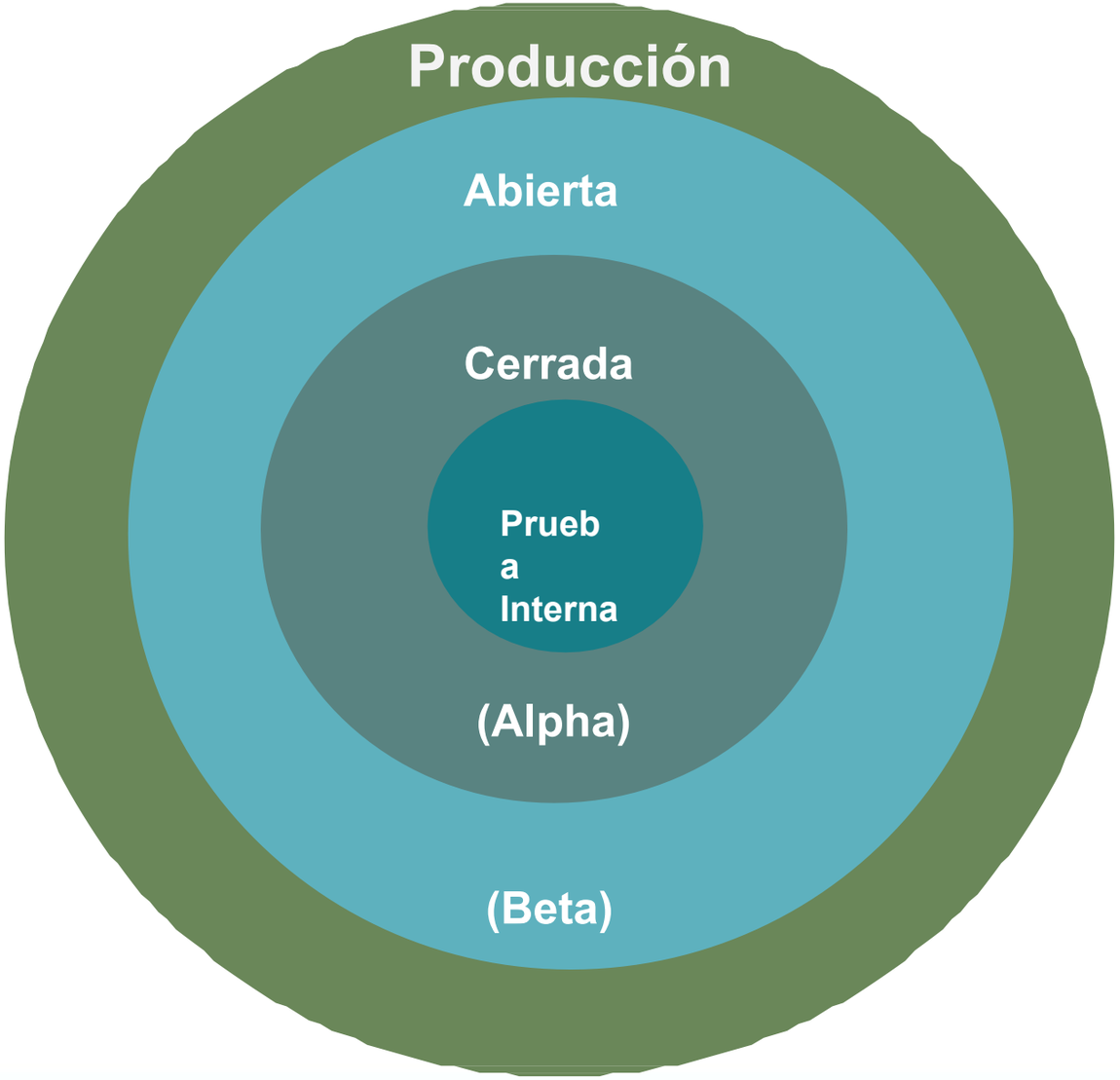
→ **Preparar servidores remotos**

- Verificar servidores

→ **Probar release app**

- Dispositivo físico
- Pantallas
- Rendimiento
- Batería

CANALES DE DISTRIBUCIÓN



- Interna: Para grupo cerrado de **Desarrollo**
- Cerrada: Grupo cerrado **público** de pruebas
- Abierta: Grupo abierto de **público**
- Producción: Para **todo** el **público**

GOOGLE PLAY

Primeros pasos
Brinda información sobre la app y configura la ficha de Play Store
Ocultar tareas
CUÉNTANOS DE QUÉ SE TRATA EL CONTENIDO DE LA APP
Acceso a apps >
Anuncios >
Clasificación de contenido >
Público objetivo >
ADMINISTRA CÓMO SE ORGANIZA Y PRESENTA LA APP
Selecciona la categoría de app y proporcionar detalles de contacto >
Configura la ficha de Play Store >

#Working Time

Sigamos ejercitando...

¡Antes de cerrar la clase! Te invitamos a: 👇👇👇

1. Repasar nuevamente la grabación de esta clase
2. Revisar el material compartido en la plataforma de Moodle (lo que se vio en clase y algún ejercicio adicional)
   a. Manual “ Distribución del aplicativo Android” (1-14)
3. Traer al próximo encuentro, todas tus dudas y consultas para verlas antes de iniciar nuevo tema.

< ¡Muchas gracias! >

alkemy